

Reviewer Pack — Minimal Order of Lie Algebroid Structures and Resolution Pro...

Entry 848833b96499 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 12:54:34 UTC

Minimal Order of Lie Algebroid Structures and Resolution Proof Width

Entry ID: 848833b96499 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 12:54:34 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Lie Algebroids) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of a Lie algebroid structure on the space of clauses of φ is linearly correlated with its resolution proof width $w(\varphi)$, such that the order of the Lie algebroid $O(\text{LieAlgebroid}(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Lie algebroids provide a generalization of vector bundles and can be used to encode geometric and algebraic structures. If the minimal order of a Lie algebroid structure on the clause space is correlated with resolution proof width, it could reveal a deeper connection between algebraic geometry and complexity theory.

Taxonomy category: Lie Theory $\times$ Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3832d10c0f975084

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$ CNFs with resolution proof width $w(\varphi)$, the ratio of the minimal order $O(\text{LieAlgebroid}(\varphi))$ to $w(\varphi)$ is bounded by a constant factor, and falsified if this ratio exceeds the constant or if any seed produces a ratio > 2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Lie algebroids' AND 'resolution proof complexity' AND 'minimal order' - 'resolution proof width' IN title OR abstract AND 'Lie algebroid structures' - correlation 'order of Lie algebroid' WITH 'resolution proof width'

Top relevant hits considered: - [http://arxiv.org/abs/0806.0919v3] Calculus on Lie algebroids, Lie groupoids and Poisson manifolds - [http://arxiv.org/abs/1212.5380v2] On properties of principal elements of Frobenius Lie algebras - [http://arxiv.org/abs/2401.05274v3] On Complex Lie Algebroids with Constant Real Rank - [http://arxiv.org/abs/1603.04490v2] Lie algebroids, gauge theories, and compatible geometrical structures - [http://arxiv.org/abs/1302.5915v4] Abstract commensurators of lattices in Lie groups - [http://arxiv.org/abs/2412.19697v2] Differentiable groupoid objects and their abstract Lie algebroids - [http://arxiv.org/abs/2304.00579v2] Hamiltonian mechanics and Lie algebroid connections - [http://arxiv.org/abs/1311.1147v1] Intersection between the geometry of generalized Lie algebroids and some aspects of interior and exterior differential s

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n - 1):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if not any(clause[i] == -clause[j] for i in range(n) for j in range(i+1, n)):
                clauses.append(clause)
        return clauses

    def compute_lie_algebroid_order(cnf):
        m = len(cnf)
        n = len(cnf[0])
        matrix = [[0] * (n + 1) for _ in range(m)]
        for i, clause in enumerate(cnf):
            for lit in clause:
                if lit > 0:
                    matrix[i][lit - 1] += 1
                else:
                    matrix[i][-1] -= 1
        rank = 0
```

```

    for row in matrix:
        if any(row[j] != 0 for j in range(n + 1)):
            rank += 1
    return rank

def resolution_width(cnf):
    queue = cnf[:]
    seen = set()
    while queue:
        clause = queue.pop(0)
        for lit in clause:
            neg_lit = -lit
            if neg_lit in seen:
                continue
            seen.add(neg_lit)
            new_clauses = []
            for other_clause in queue:
                if neg_lit in other_clause:
                    new_clause = [l for l in other_clause if l != neg_lit]
                    if new_clause and all(lit not in new_clause for lit in clause):
                        new_clauses.append(new_clause)
            queue.extend(new_clauses)
    return len(queue) + 1

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cnf = generate_cnf(n)
    lie_algebroid_order = compute_lie_algebroid_order(cnf)
    width = resolution_width(cnf)
    results.append({
        "n": n,
        "lie_algebroid_order": lie_algebroid_order,
        "width": width
    })

if not results:
    return {
        "metric_name": "lie_algebroid_order_over_width",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

lie_algebroid_orders = [r["lie_algebroid_order"] for r in results]
widths = [r["width"] for r in results]

if any(w == 0 for w in widths):
    return {
        "metric_name": "lie_algebroid_order_over_width",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
    }

```

```

        "conjecture_holds": False,
        "counterexample": "width_zero"
    }

ratio = [lie_algebroid_orders[i] / widths[i] for i in range(len(lie_algebroid_orders))]

if any(r > 2 for r in ratio):
    return {
        "metric_name": "lie_algebroid_order_over_width",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": False,
        "counterexample": f"ratio_exceeds_2 at n={results[ratio.index(max(ratio))]['n']}"
    }

    return {
        "metric_name": "lie_algebroid_order_over_width",
        "metric_value": sum(ratio) / len(ratio),
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "ratio_exceeds_2"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14489
2	propose	ollama_remote	glm4:latest	0	0	14360
3	propose	ollama_remote	glm4:latest	0	0	19156
4	propose	ollama_remote	glm4:latest	0	0	12373
5	preregistration	ollama_remote	glm4:latest	0	0	11655
6	novelty	ollama_remote	glm4:latest	0	0	8571
7	novelty	ollama_remote	glm4:latest	0	0	10884
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52008
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12256
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11946
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13306
12	judge	ollama_remote	glm4:latest	0	0	21663

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 202668 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/848833b96499.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/848833b96499.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/848833b96499.tar.gz` (if generated)